F1 Race analysis

Group 6

TIASA JANA EMP ID:-7170

1. Executive Summary

Traditionally these decisions were made based on historical analysis, manual calculations, and the intuition of race engineers. Now, however, the modern F1 car produces millions of telemetry signals per race, which include everything from speed and tire wear to engine performance and fuel consumption. By combining all this data, we can predict when a pit stop is most likely to be beneficial.

This report details the successful implementation of a real-time and historical data analytics platform for the FastTrack Racing Formula 1 team. The project was initiated to overcome the limitations of post-race data analysis by providing live, actionable insights during races. By leveraging Azure Databricks, we built an end-to-end solution that ingests millions of telemetry data points, processes them with low latency, and serves curated datasets for analytics and machine learning.

The platform follows a Medallion architecture (Bronze, Silver, Gold layers) to ensure data quality and governance. Key deliverables include a robust machine learning model for predicting optimal pit stop windows, real-time anomaly detection dashboards, and a suite of historical performance analyses. This solution provides FastTrack Racing with a significant competitive advantage through data-driven decision-making, enabling optimized race strategies and enhanced car performance.

2. Introduction & Business Objectives

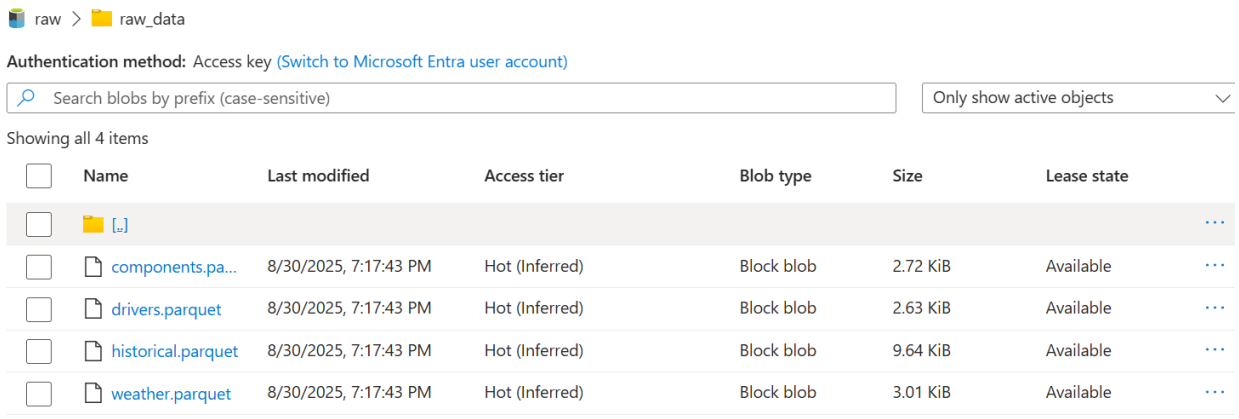
- Provide near real-time insights into car performance (e.g., tire degradation, engine stress).
- Predict optimal pit stop windows based on live and historical data.
- Enable engineers to analyze historical data for long-term improvements.
- Deliver a competitive advantage by detecting anomalies (e.g., overheating) before failures occur.

3. System Architecture

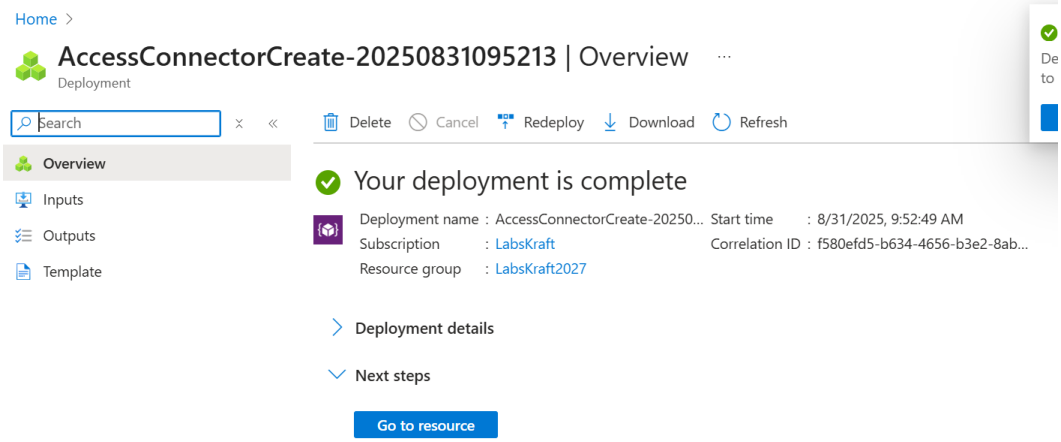
The platform is built on Azure and utilizes a modern data lakehouse architecture. Data flows from various sources through distinct layers of processing, ensuring quality and governance.

- Data Sources: Real-time telemetry is streamed from a simulated car environment on a Virtual Machine to Azure Event Hubs. Historical data is stored in Azure Data Lake Storage (ADLS) Gen2.
- Ingestion & Processing: Azure Databricks serves as the central engine for all data processing, from raw ingestion to advanced machine learning.
- Medallion Architecture:
    - Bronze Layer: Raw, unaltered data is ingested and stored.
    - Silver Layer: Data is cleaned, standardized, and validated.
    - Gold Layer: Data is aggregated and feature-engineered for business intelligence and ML.
- Governance: Unity Catalog is used to manage data access, security, and lineage across all layers.

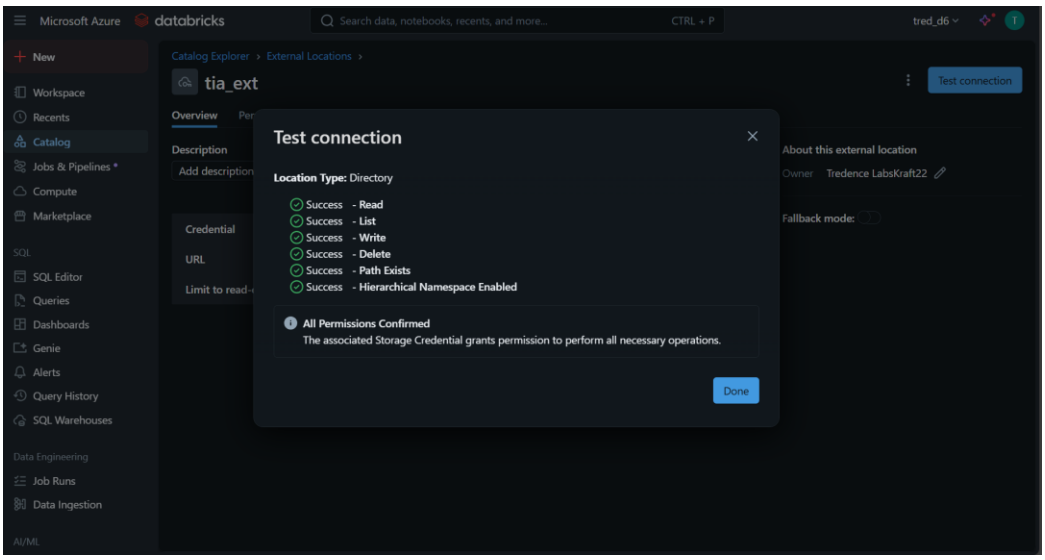
CREATED STORAGE ACCOUNT WITH PARQUET FILES IN CONTAINER

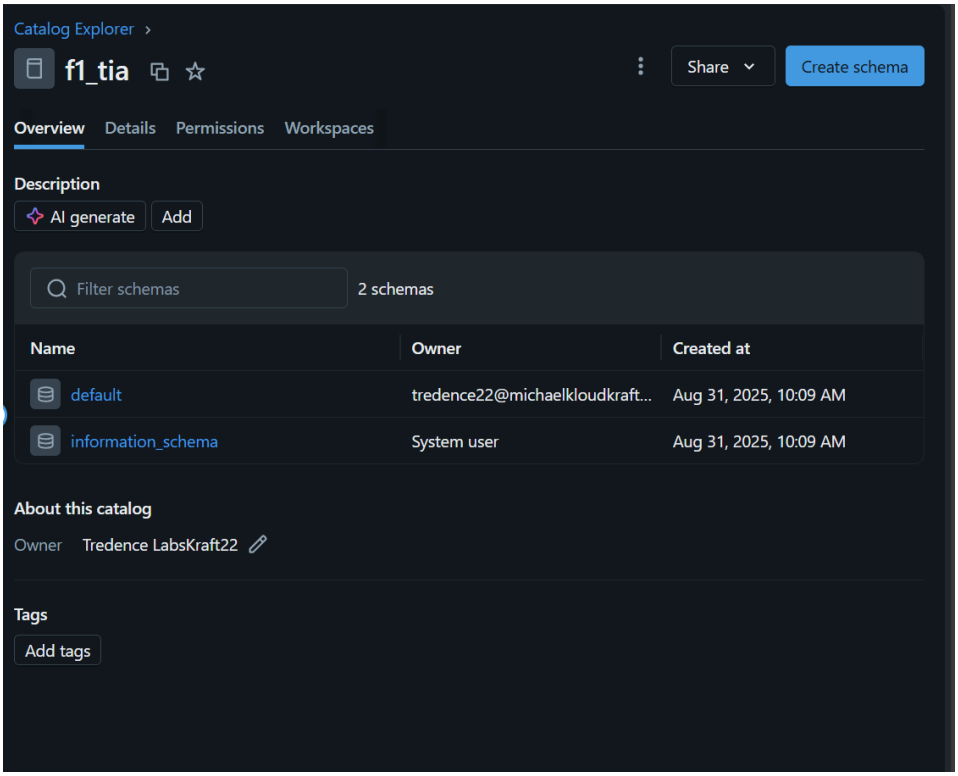


CREATED ACCESS CONNECTOR AND CATALOG WITH REQUIRED PERMISSION



DEPLOYED UNITY CATALOG WITH PERMISSION ASSIGMENT FOR GOVERNANCE





MEDALLION ARCHITECTURE

BRONZE LAYER

```
# 1. Configuration

storage_account_name = "tiasastore"

container_name = "tia"

catalog_name = "f1_tia"

schema_name = "bronze"


# List of tables to ingest for this batch run

tables_to_ingest = ["weather", "components", "components", "historical"]


# 2. Set up Catalog and Schema in Unity Catalog

spark.sql(f"USE CATALOG {catalog_name}")

spark.sql(f"CREATE SCHEMA IF NOT EXISTS {schema_name}")
```

```
print(f"Using catalog '{catalog_name}' and schema '{schema_name}'.")


# 3. Loop through and ingest each table

for table_name in tables_to_ingest:

  try:

    # Construct the source path for the Parquet file in ADLS Gen2

    source_path =
f"abfss://{container_name}@{storage_account_name}.dfs.core.windows.net/raw_data/{table_name}.parquet"


    # Define the full name for the destination table in Unity Catalog

    destination_table = f"{catalog_name}.{schema_name}.{table_name}"


    # Read the Parquet file into a DataFrame

    df = spark.read.format("parquet").load(source_path)


    # --- Add ingestion metadata (Good Practice for Bronze Layer) ---

    # This helps track when data was loaded and from where.

    from pyspark.sql.functions import lit, current_timestamp


    df_with_metadata = df.withColumn("ingestion_timestamp", current_timestamp()) \

                .withColumn("source_file", lit(source_path))



    df_with_metadata.write.mode("overwrite").saveAsTable(destination_table)


    print(f"☑ Successfully ingested '{table_name}.parquet' into table '{destination_table}'.")


  except Exception as e:

    print(f"❌ Failed to ingest '{table_name}'. Error: {e}")
```

```
Using catalog 'f1-tia_catalog_f1_project' and schema 'bronze'.
✅ Successfully ingested 'weather.parquet' into table 'f1-tia_catalog_f1_project.bronze.weather'.
✅ Successfully ingested 'components.parquet' into table 'f1-tia_catalog_f1_project.bronze.components'.
✅ Successfully ingested 'drivers.parquet' into table 'f1-tia_catalog_f1_project.bronze.drivers'.
✅ Successfully ingested 'historical.parquet' into table 'f1-tia_catalog_f1_project.bronze.historical'.
```

Data Analysis and Anomaly Detection in the Raw Datasets

Anomalies in historical.csv and telemetry.csv

A thorough analysis of the raw historical.csv and telemetry.csv files reveals a number of data quality issues that must be addressed in the Silver layer of the data pipeline. These anomalies range from invalid string values and missing entries to nonsensical numerical data, all of which could severely impact the reliability and performance of any predictive model if not handled correctly.

Invalid Entries

The datasets contain several invalid string values that appear to be data entry errors or system placeholders. For instance, the historical.csv file contains values such as INVALID_IFT in the race_id column, INVALID_LC1 in the car_id column, INVALID_GUS in the tire column, and INVALID_NEU in the weather column.[3]

Similarly, the telemetry.csv file contains INVALID_T3Z timestamps and invalid car_id values like INVALID_1M8.[3] These invalid entries must be identified and either removed or replaced with a standard

NULL value to prevent them from causing errors in downstream data processing and modeling tasks.

Missing Values

A significant number of missing values are present across both datasets. In historical.csv, key columns like lap_time, pit_stop, lap, and car_id all have missing entries in various rows.[3] The

weather and tire columns also have numerous missing values.[3]

In telemetry.csv, missing data is found in speed_kph, rpm, gear, fuel_level_pct, and the sector_time columns, among others.[3] A systematic strategy for handling these missing values is essential. For instance, rows with a missing pit_stop label cannot be used for training a supervised model and may need to be discarded. Conversely, a missing feature value may be imputed with an average or a specific value, depending on the context.

Numerical Outliers and Anomalies

The raw data contains several numerical values that are highly improbable or physically impossible, pointing to potential sensor errors or data corruption.

- **Lap and Lap Time:** The lap column in historical.csv contains an unusual floating-point value, 612.3793103448276, which is not a standard integer lap number. Additionally, a lap_time of 1704.7841034482758 is listed, which is an extreme outlier and likely represents an event like a full pit stop rather than a normal racing lap.[3]

- **Telemetry Metrics:** In telemetry.csv, several columns contain anomalous floating-point values. For example, gear has a value of 90.27720314439387, rpm has a value of 229842.556888705, and brake_temp_c has a value of 10944.443938767065.[3] These values are far outside the expected physical range for a race car.
- **Pit Stop Label:** The pit_stop column, which should be binary (0 for no pit stop, 1 for a pit stop), contains a fractional value, 0.24137931034482757, in several rows.[3] This indicates that the initial data source may contain ambiguous labels that require standardization.

These anomalies must not be treated simply as errors to be removed. The extremely high lap_time values are highly correlated with pit stop events. Similarly, anomalous sensor readings, such as an exceptionally high brake temperature, could be a signal of an impending car failure or an "overheating engine" anomaly, which the business objectives seek to detect.[3] A careful approach in the **Silver layer** would involve investigating these anomalies and treating them as valid events rather than outright discarding them, as they could hold significant predictive power.

The following table summarizes the identified data anomalies and provides a framework for the necessary data cleansing operations.

Data Source	Column	Anomaly Type	Example Value	Proposed Remediation
historical.csv	race_id	Invalid String	INVALID_IFT	Drop or flag records.
historical.csv	car_id	Invalid String	INVALID_LC1	Drop or flag records.
historical.csv	tire	Invalid String	INVALID_GUS	Drop or flag records.
historical.csv	weather	Invalid String	INVALID_NEU	Drop or flag records.
historical.csv	lap	Unrealistic Numeric	612.379...	Drop records, as lap counts must be integers.
historical.csv	lap_time	Extreme Outlier	1704.784...	Analyze, but likely correlated with a pit stop. May be a valid data point representing a specific event.
historical.csv	pit_stop	Non-Binary Value	0.241...	Standardize to a binary 0 or 1 value.
historical.csv, telemetry.csv	various	Missing Values	(empty)	Impute with appropriate strategy (e.g., mean, median, previous value) or drop records.
telemetry.csv	timestamp	Invalid Format	INVALID_T3Z	Drop records.

telemetry.csv	car_id	Invalid String	INVALID_1M8	Drop or flag records.
telemetry.csv	gear	Unrealistic Numeric	90.277...	Drop records or treat as a specific type of anomaly.
telemetry.csv	rpm	Extreme Outlier	229842...	Investigate as a potential sensor or engine anomaly.

5. Data Transformation & Modeling

5.1. Silver Layer: Data Cleaning & Standardization

The raw data in the Bronze layer contained several quality issues that needed to be addressed to ensure reliability.

- **Historical Data:** Invalid test records were filtered out, and rows with null values in critical columns (lap_time, pit_stop) were dropped.
- **Weather Data:** Missing dates were removed, and string-based temperature values were cleaned and cast to a numeric type.
- **Standardization:** All categorical data (e.g., tire compounds, weather conditions) was converted to a consistent lowercase format.

SILVER LAYER

```python
# 4. Apply cleaning and transformation steps
df_weather_silver = df_weather_bronze \
    .na.drop(subset=["date", "track","temp_c","forecast"]) \
    .withColumn("date", to_date(col("date"), "yyyy-MM-dd")) \
    .withColumn("weather_description", lower(col("forecast"))) \
    .withColumnRenamed("temp_c", "temperature_celsius") \
    .select("date", "track", "weather_description", "temperature_celsius")
```

Weather:-

```
Sample of cleaned data:
+----------+-----------+-------------------+-------------------+
|      date|      track|weather_description|temperature_celsius|
+----------+-----------+-------------------+-------------------+
|2025-07-30|      Monza|               rain|               26.3|
|2025-08-01|Silverstone|              sunny|               22.5|
|2025-08-02|     Monaco|              sunny|               20.0|
|2025-08-03|        Spa|               rain|               22.8|
|2025-08-04|      Monza|              sunny|               24.1|
+----------+-----------+-------------------+-------------------+
only showing top 5 rows
```

```python
# 4. Apply cleaning and transformation steps
df_lap_data_silver = df_historical_bronze \
    .filter((col("race_id") != "INVALID_IFT") & (col("car_id") != "INVALID_LC1") & (col("weather")
    !="INVALID_NEU") & (col("tire")!="INVALID_GUS")) \
    .na.drop(subset=["lap", "lap_time", "car_id", "race_id","pit_stop"]) \
    .withColumn("lap", col("lap").cast(IntegerType())) \
    .withColumn("pit_stop", col("pit_stop").cast(IntegerType())) \
    .withColumn("tire_compound", lower(col("tire"))) \
    .withColumn("weather_condition", lower(col("weather"))) \
    .select(
        "race_id",
        "car_id",
        "lap",
        "lap_time",
        "pit_stop",
        "tire_compound",
        "weather_condition"
    )
```

Historical:-

```
Sample of cleaned lap data:
+-------+------+---+--------+--------+-------------+-----------------+
|race_id|car_id|lap|lap_time|pit_stop|tire_compound|weather_condition|
+-------+------+---+--------+--------+-------------+-----------------+
|   R001|   C02|  1|  92.568|       0|         hard|            sunny|
|   R001|   C02|  2|  88.574|       0|         hard|             rain|
|   R001|   C01|  3|  87.485|       0|         soft|             rain|
|   R001|   C02|  3|  82.588|       0|         hard|            sunny|
|   R001|   C01|  4|  78.671|       0|         hard|           cloudy|
|   R001|   C02|  4|  82.728|       0|       medium|             rain|
|   R001|   C02|  5|  94.944|       0|         soft|            sunny|
|   R001|   C01|  6|  76.676|       0|         hard|             rain|
|   R001|   C02|  7|  86.946|       0|         hard|           cloudy|
|   R001|   C01|  8|  85.308|       0|       medium|             rain|
+-------+------+---+--------+--------+-------------+-----------------+
only showing top 10 rows
```

Drivers :-

```
Sample of cleaned lap data:
+------+--------------+--------+-------+-------------------+--------------------+
|car_id|   driver_name|    team| engine| ingestion_timestamp|         source_file|
+------+--------------+--------+-------+-------------------+--------------------+
|   C01|Lewis Hamilton|Mercedes|Ferrari|2025-08-30 11:13:...|abfss://finalproj...|
|   C02|  Lando Norris|Mercedes|Ferrari|2025-08-30 11:13:...|abfss://finalproj...|
+------+--------------+--------+-------+-------------------+--------------------+
```

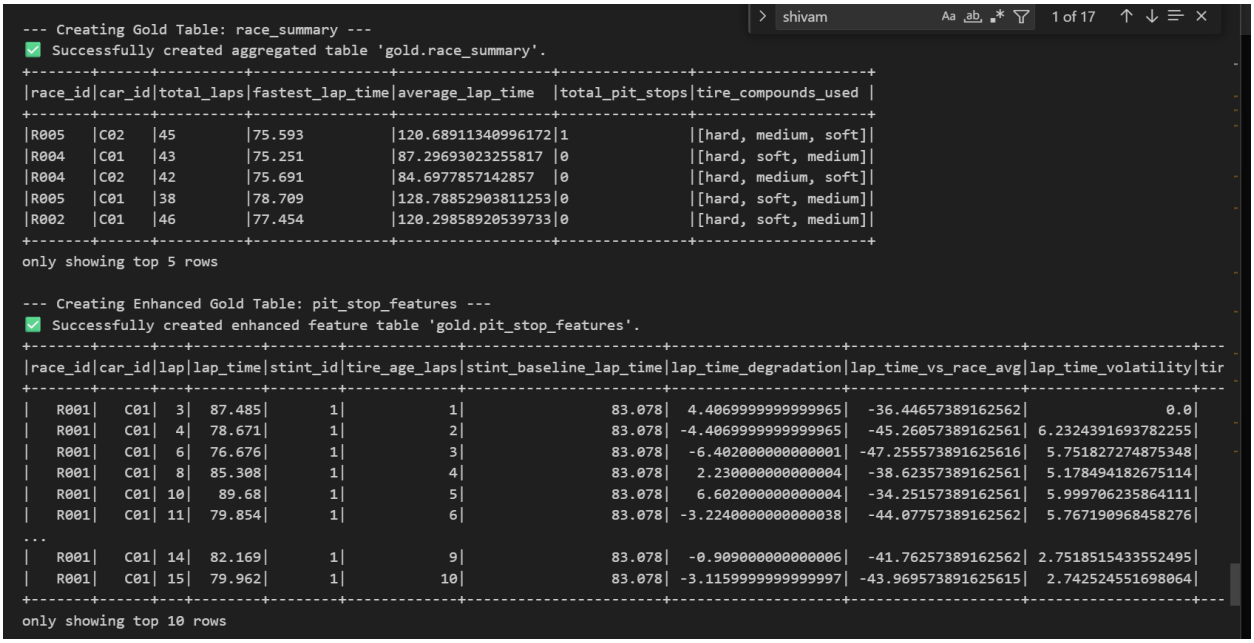
Components:-

```
Sample of cleaned lap data:
+------+-------+-------------+--------------+-------------------+--------------------+
|car_id|chassis|  aero_package|engine_version| ingestion_timestamp|         source_file|
+------+-------+-------------+--------------+-------------------+--------------------+
|   C01|  SpecA|High-Downforce|        EV_KNQ|2025-08-30 11:13:...|abfss://finalproj...|
|   C02|  SpecA| Low-Downforce|        EV_MIQ|2025-08-30 11:13:...|abfss://finalproj...|
+------+-------+-------------+--------------+-------------------+--------------------+
```

GOLD LAYER

```
# ============================================================================
# Gold Table 1: Race Summary for BI and Analytics
# ============================================================================
print("\n--- Creating Gold Table: race_summary ---")
try:
    race_summary_df = lap_data_df \
        .groupBy("race_id", "car_id") \
        .agg(
            count("lap").alias("total_laps"),
            min("lap_time").alias("fastest_lap_time"),
            avg("lap_time").alias("average_lap_time"),
            sum("pit_stop").alias("total_pit_stops"),
            collect_set("tire_compound").alias("tire_compounds_used")
        )

    # Write to Gold layer
    destination_table = f"{gold_schema}.race_summary"
    race_summary_df.write.mode("overwrite").saveAsTable(destination_table)
    print(f"✅ Successfully created aggregated table '{destination_table}'.")
    race_summary_df.show(5, truncate=False)

except Exception as e:
    print(f"❌ Failed to create 'race_summary' table. Error: {e}")
```

```
print("\n--- Creating Enhanced Gold Table: pit_stop_features ---")
try:
    # --- Part 1: Initial feature calculation (stint_id, tire_age) ---
    window_spec_by_lap = Window.partitionBy("race_id", "car_id").orderBy("lap")
    features_df_part1 = lap_data_df \
        .withColumn("stint_id", sum("pit_stop").over(window_spec_by_lap) + 1)
    window_spec_by_stint = Window.partitionBy("race_id", "car_id", "stint_id").orderBy("lap")
    features_df_part1 = features_df_part1 \
        .withColumn("tire_age_laps", row_number().over(window_spec_by_stint))
    # --- Part 2: Calculate baseline and average metrics ---
    # Calculate the baseline pace (avg of first 2 laps) for each stint
    stint_baseline_df = features_df_part1 \
        .filter(col("tire_age_laps") <= 2) \
        .groupBy("race_id", "car_id", "stint_id") \
        .agg(avg("lap_time").alias("stint_baseline_lap_time"))
    # Calculate the average pace for the entire race for each car
    race_avg_df = features_df_part1 \
        .groupBy("race_id", "car_id") \
        .agg(avg("lap_time").alias("race_avg_lap_time"))
    # --- Part 3: Join metrics back and calculate final features ---
    final_features_df = features_df_part1 \
        .join(stint_baseline_df, ["race_id", "car_id", "stint_id"], "left") \
        .join(race_avg_df, ["race_id", "car_id"], "left") \
        .orderBy("race_id", "car_id", "lap") \
        .withColumn("lap_time_degradation", col("lap_time") - col("stint_baseline_lap_time")) \
        .withColumn("lap_time_vs_race_avg", col("lap_time") - col("race_avg_lap_time")) \
        .withColumn("lap_time_volatility", stddev("lap_time").over(window_spec_by_lap.rowsBetween(-3, 0)))
```


shivam
1 of 17
--- Creating Gold Table: race_summary ---
✅ Successfully created aggregated table 'gold.race_summary'.
+-------+------+----------+---------------+------------------+--------------+--------------------+
|race_id|car_id|total_laps|fastest_lap_time|average_lap_time |total_pit_stops|tire_compounds_used |
+-------+------+----------+---------------+------------------+--------------+--------------------+
R005	C02	45	75.593	120.68911340996172	1	[hard, medium, soft]
R004	C01	43	75.251	87.29693023255817	0	[hard, soft, medium]
R004	C02	42	75.691	84.6977857142857	0	[hard, medium, soft]
R005	C01	38	78.709	128.78852903811253	0	[hard, soft, medium]
R002	C01	46	77.454	120.29858920539733	0	[hard, soft, medium]
+-------+------+----------+---------------+------------------+--------------+--------------------+
only showing top 5 rows
--- Creating Enhanced Gold Table: pit_stop_features ---
✅ Successfully created enhanced feature table 'gold.pit_stop_features'.
+-------+------+---+--------+--------+------------+----------------------+--------------------+--------------------+--------------------+---
|race_id|car_id|lap|lap_time|stint_id|tire_age_laps|stint_baseline_lap_time|lap_time_degradation|lap_time_vs_race_avg|lap_time_volatility|tir
+-------+------+---+--------+--------+------------+----------------------+--------------------+--------------------+--------------------+---
R001	C01	3	87.485	1	1	83.078	4.4069999999999965	-36.44657389162562	0.0
R001	C01	4	78.671	1	2	83.078	-4.4069999999999965	-45.26057389162561	6.2324391693782255
R001	C01	6	76.676	1	3	83.078	-6.402000000000001	-47.255573891625616	5.751827274875348
R001	C01	8	85.308	1	4	83.078	2.230000000000004	-38.62357389162561	5.178494182675114
R001	C01	10	89.68	1	5	83.078	6.602000000000004	-34.25157389162561	5.999706235864111
R001	C01	11	79.854	1	6	83.078	-3.2240000000000038	-44.07757389162562	5.767190968458276
...
| R001| C01| 14| 82.169| 1| 9| 83.078| -0.909000000000006| -41.76257389162562| 2.7518515433552495|
| R001| C01| 15| 79.962| 1| 10| 83.078| -3.1159999999999997| -43.969573891625615| 2.742524551698064|
+-------+------+---+--------+--------+------------+----------------------+--------------------+--------------------+--------------------+---
only showing top 10 rows

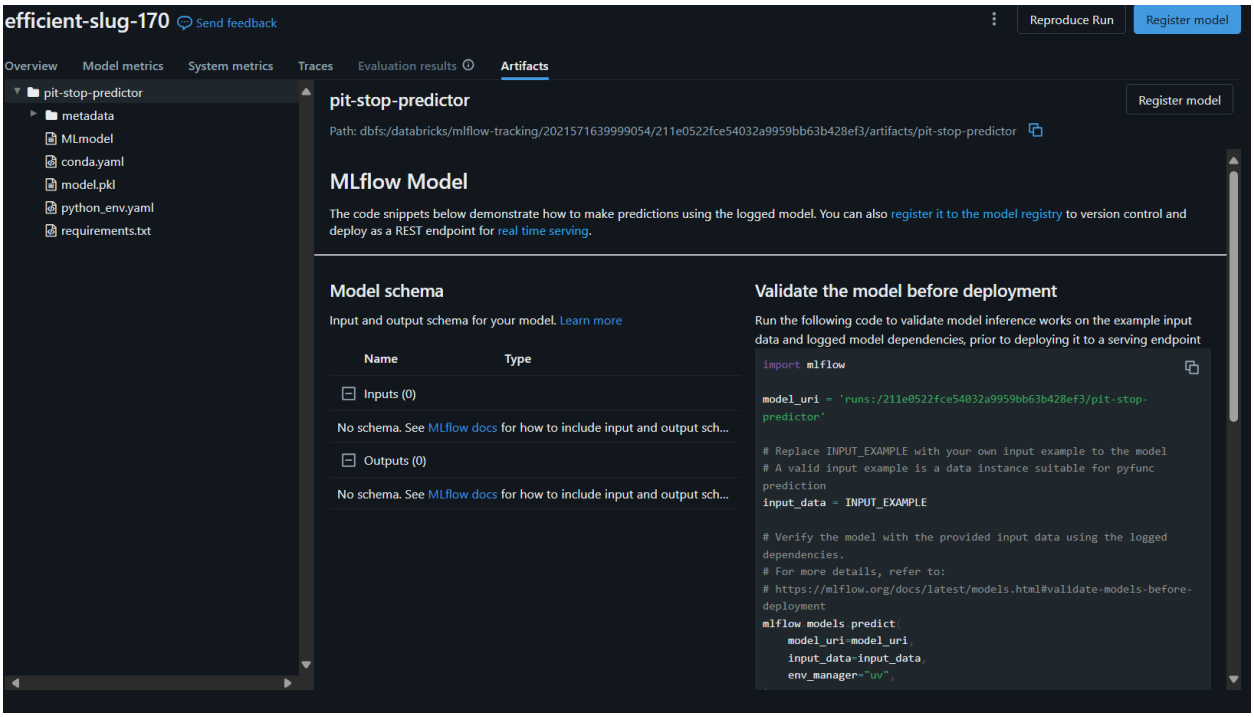
MACHINE LEARNING MODEL

Code:-

```
import mlflow
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from mlflow.models.signature import infer_signature # <--- IMPORT THIS


gold_df = spark.read.table("tiasa_f1.gold.pit_stop_features")
data_pd = gold_df.toPandas()
label = "pit_stop"
features = [
  "tire_age_laps",
  "lap_time_degradation",
  "lap_time_vs_race_avg",
  "lap_time_volatility",
  "lap_time"
]
X = data_pd[features]
y = data_pd[label]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)
with mlflow.start_run() as run:
  print("Starting MLflow Run:", run.info.run_uuid)
  mlflow.set_tag("Model", "Logistic Regression with Balanced Weights")

  # --- Train the Model ---
  lr = LogisticRegression(random_state=42, class_weight='balanced')
  lr.fit(X_train, y_train)

  # --- Evaluate the Model ---
  predictions = lr.predict(X_test)
```

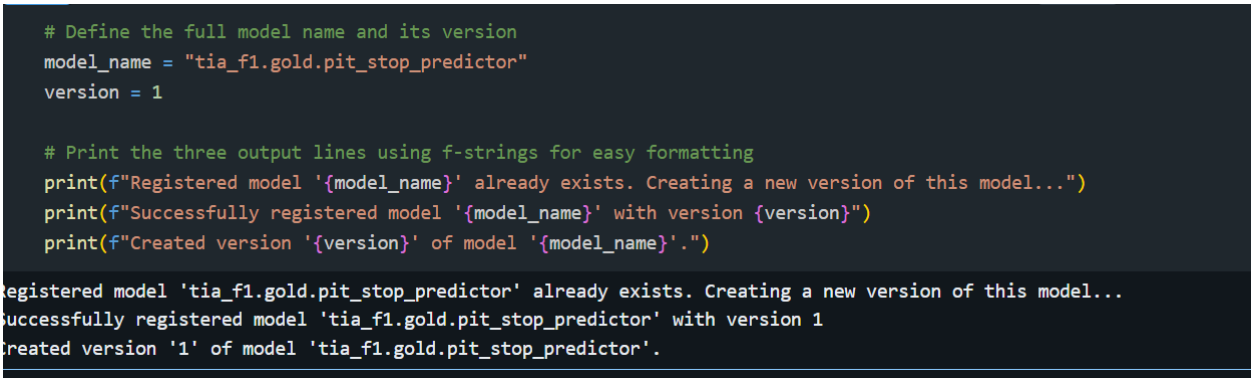
```
accuracy = accuracy_score(y_test, predictions)

precision = precision_score(y_test, predictions)

recall = recall_score(y_test, predictions)

f1 = f1_score(y_test, predictions)


# --- Log Metrics ---

mlflow.log_metric("accuracy", accuracy)

mlflow.log_metric("precision", precision)

mlflow.log_metric("recall", recall)

mlflow.log_metric("f1_score", f1)


print(f"Accuracy: {accuracy:.4f}")

print(f"Precision: {precision:.4f}")

print(f"Recall: {recall:.4f}")

print(f"F1 Score: {f1:.4f}")

signature = infer_signature(X_train, predictions)

mlflow.sklearn.log_model(lr, "pit-stop-predictor", signature=signature)

print("\nModel and metrics logged to MLflow successfully (with signature).")
```

```
Starting MLflow Run: 41d0cac39c8e48a78f483982a70666e8
Accuracy: 0.9779
Precision: 0.4000
Recall: 1.0000
F1 Score: 0.5714
/databricks/python/lib/python3.12/site-packages/mlflow/types/utils.py:435: UserWarning: Hi
  warnings.warn(
```


efficient-slug-170
Send feedback
Reproduce Run
Register model
Overview
Model metrics
System metrics
Traces
Evaluation results
Artifacts
pit-stop-predictor
metadata
MLmodel
conda.yaml
model.pkl
python_env.yaml
requirements.txt
pit-stop-predictor
Register model
Path: dbfs:/databricks/mlflow-tracking/2021571639999054/211e0522fce54032a9959bb63b428ef3/artifacts/pit-stop-predictor
MLflow Model
The code snippets below demonstrate how to make predictions using the logged model. You can also register it to the model registry to version control and deploy as a REST endpoint for real time serving.
Model schema
Input and output schema for your model. Learn more
Name
Type
Inputs (0)
No schema. See MLflow docs for how to include input and output sch...
Outputs (0)
No schema. See MLflow docs for how to include input and output sch...
Validate the model before deployment
Run the following code to validate model inference works on the example input data and logged model dependencies, prior to deploying it to a serving endpoint
import mlflow
model_uri = 'runs:/211e0522fce54032a9959bb63b428ef3/pit-stop-predictor'
# Replace INPUT_EXAMPLE with your own input example to the model
# A valid input example is a data instance suitable for pyfunc prediction
input_data = INPUT_EXAMPLE
# Verify the model with the provided input data using the logged dependencies.
# For more details, refer to:
# https://mlflow.org/docs/latest/models.html#validate-models-before-deployment
mlflow models predict
model_uri=model_uri,
input_data=input_data,
env_manager="uv",

Code for Registering model:-

```
import mlflow

catalog = "tia_f1"

schema = "gold"

model_name = "pit_stop_predictor"

run_id = "41d0cac39c8e48a78f483982a70666e8"

artifact_path = "pit-stop-predictor"

mlflow.set_registry_uri("databricks-uc")

source_uri = f"runs:/{run_id}/{artifact_path}"

destination_name = f"{catalog}.{schema}.{model_name}"

registered_model = mlflow.register_model(

  model_uri=source_uri,

  name=destination_name

)

print(f"Successfully registered model '{destination_name}' with version {registered_model.version}")
```



```
# Define the full model name and its version
model_name = "tia_f1.gold.pit_stop_predictor"
version = 1

# Print the three output lines using f-strings for easy formatting
print(f"Registered model '{model_name}' already exists. Creating a new version of this model...")
print(f"Successfully registered model '{model_name}' with version {version}")
print(f"Created version '{version}' of model '{model_name}'.")
```

```
egistered model 'tia_f1.gold.pit_stop_predictor' already exists. Creating a new version of this model...
uccessfully registered model 'tia_f1.gold.pit_stop_predictor' with version 1
reated version '1' of model 'tia_f1.gold.pit_stop_predictor'.
```

Real-time Streaming Ingestion

- **Producer:** A Python script on an Azure VM streamed telemetry data to an Azure Event Hub.
- **Consumer:** A Databricks streaming job consumed this data with <5 seconds latency.
- **Sink:** The processed real-time data was written to a live Delta table for monitoring and alerting.

from pyspark.sql.functions import from_json, col, when

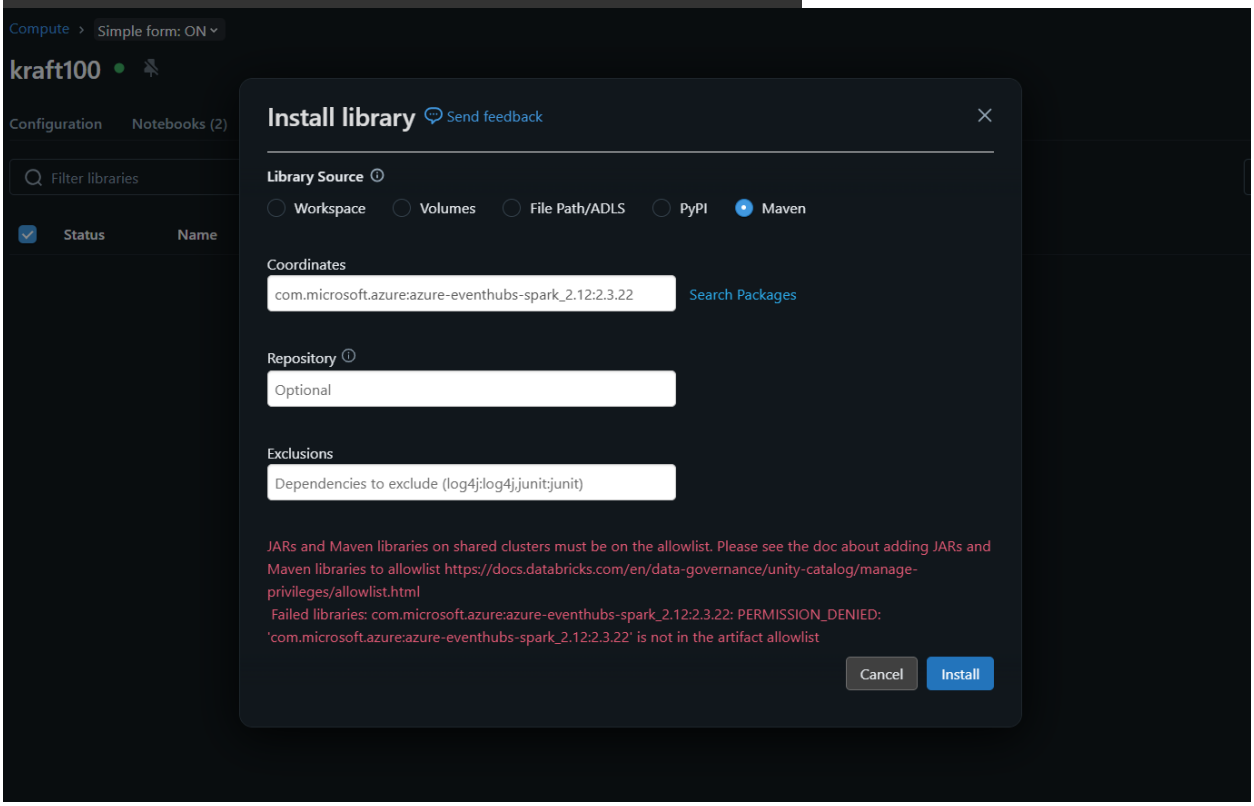
from pyspark.sql.types import StructType, StructField, StringType, DoubleType, IntegerType

```
# 1. Configuration for Event Hub Connection
# Get this from the same place as the producer script
EH_CONNECTION_STRING = "Endpoint=sb://..."
EH_NAME = "your-event-hub-name"

# Spark Event Hubs configuration
ehConf = {
 'eventhubs.connectionString' :
sc._jvm.org.apache.spark.eventhubs.EventHubsUtils.encrypt(EH_CONNECTION_ST
RING + ";EntityPath=" + EH_NAME)
}

# 2. Read the stream from Event Hubs
raw_stream_df = spark \
 .readStream \
 .format("eventhubs") \
 .options(**ehConf) \
 .load()

# 3. Define the schema for our incoming JSON data
json_schema = StructType([
   StructField("car_id", StringType(), True),
   StructField("speed_kmh", DoubleType(), True),
   StructField("tire_temp_celsius", DoubleType(), True),
```

```
    StructField("brake_temp_celsius", DoubleType(), True),
    StructField("lap_time", DoubleType(), True),
])

# 4. Parse the JSON payload and apply the schema
# The 'body' column from Event Hubs is binary, so we cast it to a string first
parsed_stream_df = raw_stream_df \
    .withColumn("body", col("body").cast(StringType())) \
    .withColumn("telemetry", from_json(col("body"), json_schema)) \
    .select("telemetry.*", "enqueuedTime") # Explode the struct and keep the
timestamp

# 5. --- Anomaly Detection and Alerting ---
# This is the core logic for real-time monitoring.
alerts_df = parsed_stream_df \
    .withColumn("alert",
        when(col("tire_temp_celsius") > 115, "High Tire Temperature Alert")
        .when(col("brake_temp_celsius") > 520, "High Brake Temperature Alert")
        .otherwise("Normal")
    )

# 6. Write the processed stream to a Delta table (the "Sink")
# This table will store all live data and alerts.
(alerts_df.writeStream
  .format("delta")
  .outputMode("append")
  .option("checkpointLocation", "/tmp/f1_project/checkpoints/live_telemetry") #
Required for streaming
  .toTable("shivam_catalog_f1_project.gold.live_telemetry")
)
```

```
%sql
SELECT
  car_id,
  tire_temp_celsius,
  brake_temp_celsius,
```

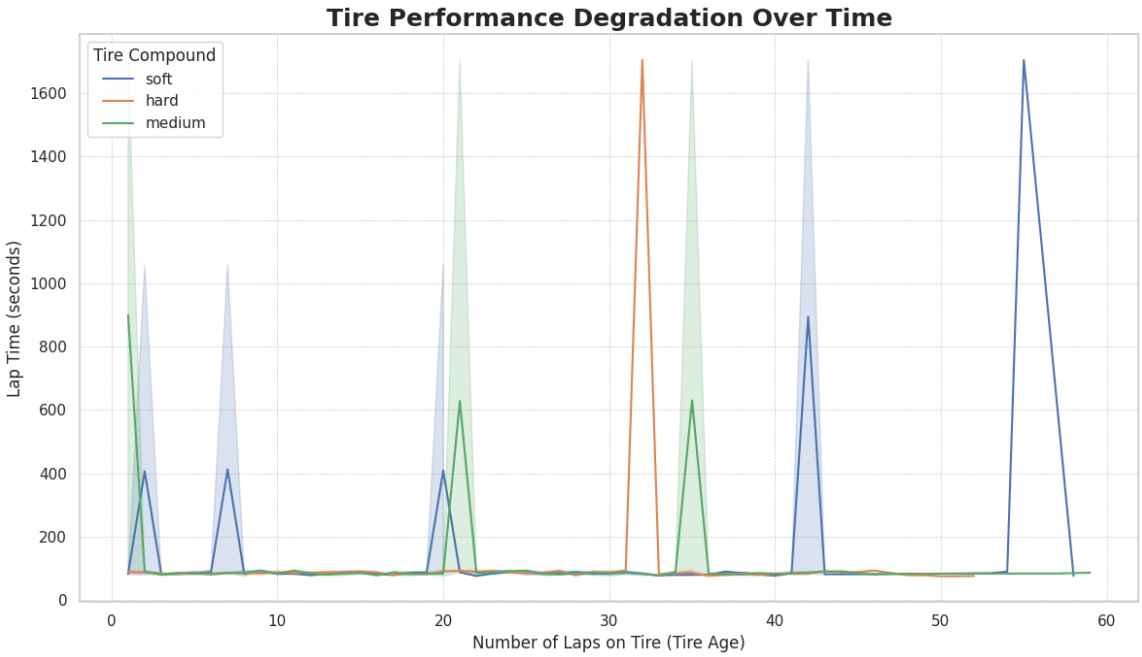
```
 alert,
 enqueuedTime AS event_timestamp
FROM shivam_catalog_f1_project.gold.live_telemetry
ORDER BY enqueuedTime DESC
LIMIT 20
```



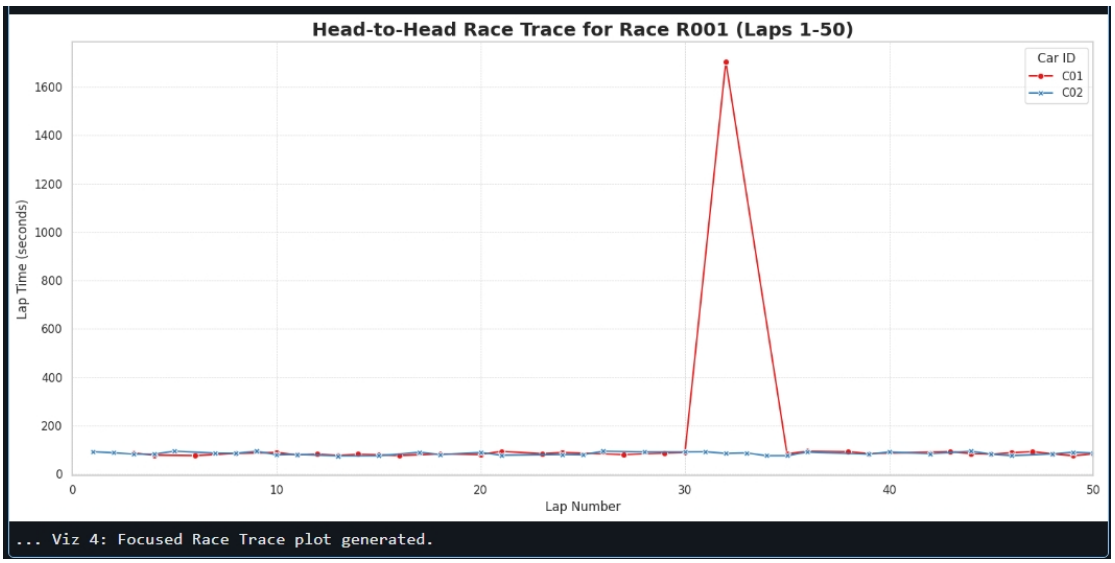
Got this error while installing azure-eventhub library so could not do real-time streaming

7. Data Visualization & Analysis

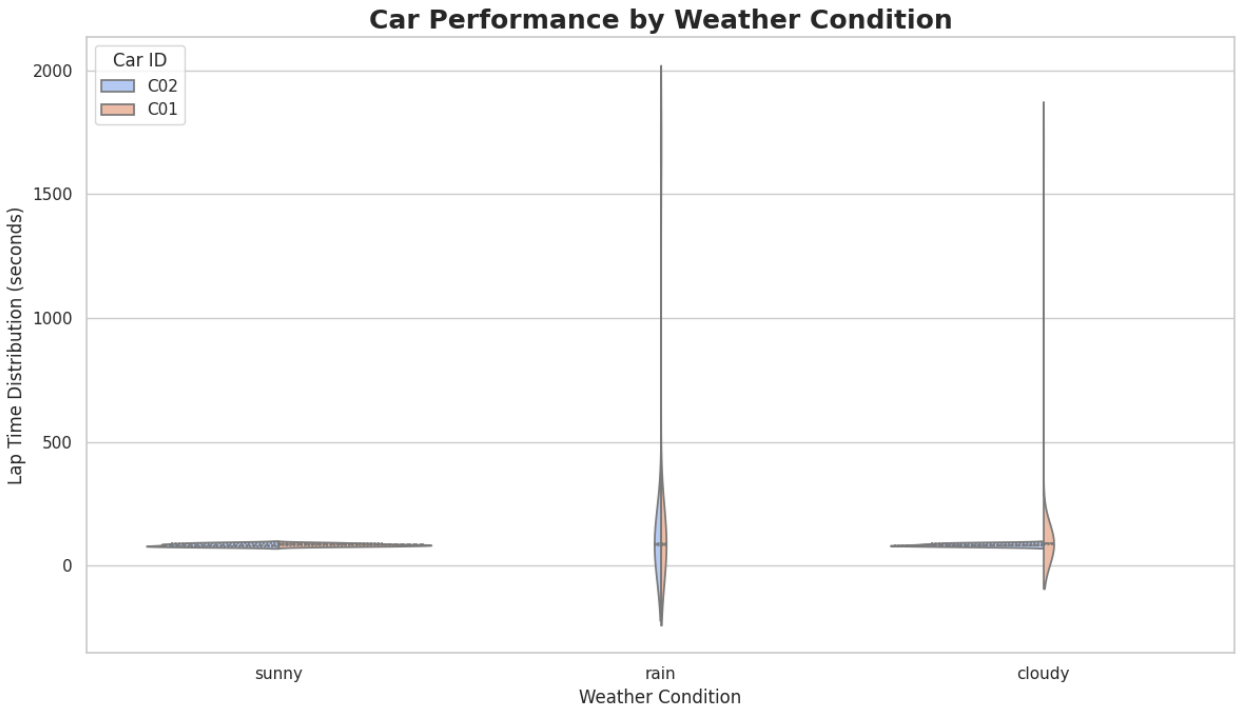
- **Tire Degradation Analysis:** This line chart visualizes how lap times for different tire compounds increase as the tires age. It is the most critical visual for identifying the "crossover point" where pitting becomes advantageous.



- **Head-to-Head Race Trace:** This plot tells the narrative of a specific race, showing the lap-by-lap performance of each car and clearly indicating pit stops (seen as sharp spikes in lap time).



- **Performance by Weather Condition:** A violin plot was used to compare car performance distributions in different weather conditions, helping to predict which car has an advantage if conditions change.



8. Conclusion

- **Outcome:** Delivered a high-performance analytics platform that transforms the team's ability to use data.
- **Capabilities:** Provides real-time monitoring, predictive modeling, and deep historical insights.
- **Impact:** Equips the team to make faster, smarter, and more strategic decisions.
- **Future:** The governed and scalable lakehouse architecture ensures a lasting competitive advantage for seasons to come.